

IcenterLunarRoverChassisControlDemo API 接口说明书

----- V1.0.0

----- 王笑

本文档对应 main.py 的 idle 示例区，说明其中 12 个常用控制接口的功能、参数、返回值和调试注意事项。

1. 接口总览

1. 接口总览	1
2. drive 底盘行驶接口	2
3. stop 底盘停车接口	3
4. set_camera_angle 相机角度接口	4
5. apply_initial_pose 机械臂回初始位接口	4
6. jog_joints 机械臂关节点动接口	5
7. jog_camera 相机点动接口	7
8. motor_bus.set_acc 和 motor_bus.set_speed 电机直控接口	9
9. set_steering_angles 六个转向舵机直控接口	10
10. center_chassis_servos 转向舵机回正接口	11
11. set_arm_joint_angles 机械臂四关节绝对角接口	11
12. read_arm_joint_angles 机械臂四关节读角接口	12
13. 预留舵机接口	13
14. pivot_turn 底盘原地转向接口	14
15. 接口选择建议	15
16. 常见问题	16

2. drive 底盘行驶接口

接口位置: chassis_control.py, LunarRover.drive

调用形式:

rover.drive(speed_rad_s, steer_angle_deg)

功能:

- 同时控制四个驱动电机和六个转向舵机。
- 用于普通前进、后退和带转向角行驶。
- 示例 1 和示例 2 都使用这个接口。

参数:

参数	类型	单位	默认值	说明
speed_rad_s	float	rad/s	必填	车轮目标角速度。正数前进，负数后退，0 表示停止驱动。
steer_angle_deg	float	deg	必填	底盘转向角。0 表示直行，正负表示两个转向方向。

限制范围:

项目	当前范围
speed_rad_s	由 MAX_MOTOR_RPM=200.0 限制，约为 -20.94 到 20.94 rad/s
steer_angle_deg	-90.0 到 90.0 deg

返回值:

- 无显式返回值。

内部行为:

- **steer_angle_deg** 会先限幅，再按舵机安装方向转换后下发。
- 每次行驶会给四个电机设置默认加速度 **DEFAULT_ACC_RAD_S2**。
- 程序内部带有命令缓存，转向角、电机速度、加速度不变时会跳过重复下发。

```
# 示例 1: 底盘以 2.0 rad/s 前进 1 秒, 然后停车。
print("示例 1: 底盘以 2.0 rad/s 前进 1 秒, 然后停车。")
rover.drive(speed_rad_s=2.0, steer_angle_deg=0.0)
time.sleep(1) #必须要延迟, 让指令有执行时间, 如果没有sleep, 指令会被立即覆盖, 导致小车没有动作。
rover.stop()
time.sleep(1)

# 示例 2: 底盘以 2.0 rad/s、20 度转向角前进 1 秒, 然后停车。
print("示例 2: 底盘以 2.0 rad/s、20 度转向角前进 1 秒, 然后停车。")
rover.drive(speed_rad_s=2.0, steer_angle_deg=20.0)
time.sleep(1)
rover.stop()
time.sleep(1)
```

注意事项:

- 必须要延迟, 让指令有执行时间, 如果没有 `sleep`, 指令会被立即覆盖, 导致小车没有动作。
- `drive` 是底盘推荐入口, 比直接调用 `motor_bus` 更安全。
- 调试时应先使用较小速度, 例如 1.0 到 3.0 rad/s。
- 如果只想停车, 建议调用 `rover.stop()`, 不要用 `drive(speed_rad_s=0, ...)` 代替所有停车场景。

3. stop 底盘停车接口

接口位置: `chassis_control.py`, `LunarRover.stop`

调用形式:

`rover.stop()`

功能:

- 将四个驱动电机目标速度置为 0。
- 用于 `drive` 或 `pivot_turn` 后停车。

参数:

- 无。

返回值:

- 无显式返回值。

内部行为:

- `rover` 内部记录自己是否已经停车。
- 如果 `rover` 认为电机已经停止, 会跳过重复停车命令。

注意事项:

- 如果前面的运动是通过 `rover.drive` 或 `rover.pivot_turn` 发出的, 使用 `rover.stop()` 是合适的。

• 如果前面的运动是直接通过 `rover.motor_bus.set_speed` 发出的，`rover` 的内部停车状态可能不知道电机已经运动，此时建议直接调用 `rover.motor_bus.stop_all(rover.motor_ids)`，或逐个调用 `rover.motor_bus.set_speed(motor_id, 0.0)`。

4. set_camera_angle 相机角度接口

接口位置: `servo_control.py`, `ServoControl.set_camera_angle`

调用形式:

`rover.servo_control.set_camera_angle(camera_angle)`

功能:

- 控制相机舵机转到指定角度。
- 示例 3 使用该接口把相机转到 -30 度，再回到 0 度。

参数:

参数	类型	单位	默认值	说明
<code>camera_angle</code>	<code>float</code>	<code>deg</code>	必填	相机目标角度。

限制范围:

项目	当前范围
<code>camera_angle</code>	-90.0 到 0.0 deg

返回值:

- 无显式返回值。

内部行为:

- `camera_angle` 会自动限幅到 -90.0 到 0.0 deg。
- 程序会按相机舵机安装方向转换后下发到底层舵机。

```
# 示例 3: 相机转到 -30 度，再回到 0 度。
print("示例 3: 相机转到 -30 度，再回到 0 度。")
rover.servo_control.set_camera_angle(-30)
time.sleep(1)
rover.servo_control.set_camera_angle(0)
time.sleep(1)
```

注意事项:

- 这是绝对角度控制，不是点动控制。
- 如果前面运行过相机绝对角度控制，使用 `jog_camera()` 前先调用 `rover.arm.sync_camera_from_servo()` 同步真实相机角度。

5. apply_initial_pose 机械臂回初始位接口

接口位置: `arm_control.py`, `RobotArm.apply_initial_pose`

调用形式:

功能:

- 将机械臂四个关节移动到程序定义的初始角度。

当前初始角:

关节	初始角度
Roll	0.0 deg
Pitch1	50.0 deg
Pitch2	-140.0 deg
Pitch3	0.0 deg

返回值:

返回一个字典，包含本次目标姿态:

字段	说明
ok	True 表示命令已通过校验并下发。
speed_deg_s	本次使用的运动速度。
roll_deg	Roll 目标角。
pitch1_deg	Pitch1 目标角。
pitch2_deg	Pitch2 目标角。
pitch3_deg	Pitch3 目标角。

可能异常:

- 如果初始角配置超出关节限位，会抛出 `ArmKinematicsError`。

```
# 示例 4: 机械臂回初始位。
print("示例 4: 机械臂回初始位。")
rover.arm.apply_initial_pose()
time.sleep(2) #要给机械臂足够时间回初始位，否则后续指令可能会被覆盖，导致机械臂无法到达目标位置。
```

注意事项:

- 该接口会更新 `arm_control.py` 内部维护的机械臂角度状态。

6. jog_joints 机械臂关节点动接口

接口位置: `arm_control.py`, `RobotArm.jog_joints`

调用形式:

```
rover.arm.jog_joints(roll_delta_deg=0.0, pitch1_delta_deg=0.0, pitch2_delta_deg=0.0,
pitch3_delta_deg=0.0)
```

功能:

- 在机械臂内部记录的当前角度基础上叠加 `delta`，实现点动控制。
- 示例 5 使用该接口分别点动 Roll、Pitch1、Pitch2、Pitch3。

• 使用 `jog_joints` 前，如果前面运行过绝对角度控制，先调用 `rover.arm.sync_from_servos()` 同步真实关节角。

- PS2 遥控的 L2 机械臂模式也使用该接口做连续关节点动。

参数:

参数	类型	单位	默认值	说明
roll_delta_deg	float	deg	0.0	Roll 本次增量。
pitch1_delta_deg	float	deg	0.0	Pitch1 本次增量。
pitch2_delta_deg	float	deg	0.0	Pitch2 本次增量。
pitch3_delta_deg	float	deg	0.0	Pitch3 本次增量。

关节限位:

关节	当前范围
Roll	-180.0 到 180.0 deg
Pitch1	-90.0 到 90.0 deg
Pitch2	-150.0 到 150.0 deg
Pitch3	-150.0 到 150.0 deg

返回值:

返回一个字典，包含点动后的目标姿态:

字段	说明
ok	True 表示命令已通过校验并下发。
speed_deg_s	本次使用的运动速度。
roll_deg	点动后的 Roll 目标角。
pitch1_deg	点动后的 Pitch1 目标角。
pitch2_deg	点动后的 Pitch2 目标角。
pitch3_deg	点动后的 Pitch3 目标角。

可能异常:

- 如果任一关节点动后的目标角超出限位，会抛出 `ArmKinematicsError`。
- 抛出异常时，本次目标不会下发。

示例:

```

# 示例 5: 机械臂单关节点动, 分别控制 Roll、Pitch1、Pitch2、Pitch3。
# 如果前面运行过绝对角度控制, 先同步真实舵机角度, 再做 jog 增量控制。
print("示例 5: 机械臂单关节点动, 分别控制 Roll、Pitch1、Pitch2、Pitch3。")
rover.arm.sync_from_servos()
rover.arm.jog_joints(roll_delta_deg=2.0)
time.sleep(1)
rover.arm.jog_joints(roll_delta_deg=-2.0)
time.sleep(1)
rover.arm.jog_joints(pitch1_delta_deg=2.0)
time.sleep(1)
rover.arm.jog_joints(pitch1_delta_deg=-2.0)
time.sleep(1)
rover.arm.jog_joints(pitch2_delta_deg=2.0)
time.sleep(1)
rover.arm.jog_joints(pitch2_delta_deg=-2.0)
time.sleep(1)
rover.arm.jog_joints(pitch3_delta_deg=2.0)
time.sleep(1)
rover.arm.jog_joints(pitch3_delta_deg=-2.0)
time.sleep(1)

```

使用建议:

- 点动量建议从 2.0 到 8.0 deg 开始测试。
- 需要连续遥控时, 点动量应和控制频率匹配, 避免单次命令太大导致动作跳变。
- 当前 PS2 机械臂点动使用固定步长, 写在 ps2_control.py 内部。

内部状态说明:

- jog_joints 使用的是程序内部记录的 roll_deg、pitch1_deg、pitch2_deg、pitch3_deg。
- 它不会每次都读取真实舵机角度, 因为读角会明显降低遥控循环频率。
- 如果上电后舵机实际位置和内部记录不一致, 或人为改变过舵机位置, 可调用

rover.arm.sync_from_servos() 同步一次。

7. jog_camera 相机点动接口

接口位置: arm_control.py, RobotArm.jog_camera

调用形式:

```
rover.arm.jog_camera(delta_deg)
```

功能:

- 在内部记录的相机角度基础上叠加 delta, 实现相机点动。
- 示例 6 使用该接口让相机点动后再转回。
- 使用 jog_camera 前, 如果前面运行过相机绝对角度控制, 先调用

rover.arm.sync_camera_from_servo() 同步真实相机角度。

参数:

参数	类型	单位	默认值	说明
----	----	----	-----	----

delta_deg	float	deg	必填	本次相机角度增量。
-----------	-------	-----	----	-----------

限制范围:

项目	当前范围
点动后的相机角度	-90.0 到 0.0 deg

返回值:

- 无显式返回值。

内部行为:

- 点动后的相机角度会自动限幅。
- 该接口会更新 RobotArm 内部的 camera_angle_deg。
- 它不会每次读取真实相机舵机角度。

```
# 示例 6: 相机舵机点动, 再转回。
# 如果前面运行过相机绝对角度控制, 先同步真实相机角度, 再做 jog 增量控制。
print("示例 6: 相机舵机点动, 再转回。")
rover.arm.sync_camera_from_servo() #使用jog控制前, 要先同步一次当前舵机的角度
rover.arm.jog_camera(8.0)
time.sleep(1)
rover.arm.jog_camera(-8.0)
time.sleep(1)
```

注意事项:

- 如果需要先以真实舵机角度为准, 可调用 `rover.arm.sync_camera_from_servo()` 同步一次。
- 如果只是转到某个固定角度, 使用 `rover.servo_control.set_camera_angle(angle)` 更直接。

8. motor_bus.set_acc 和 motor_bus.set_speed 电机直控接口

接口位置: motor_lib.py, MotorBus.set_acc、MotorBus.set_speed

调用形式:

```
rover.motor_bus.set_acc(motor_id, acc_rad_s2)
```

```
rover.motor_bus.set_speed(motor_id, speed_rad_s)
```

功能:

- 直接向指定 CAN 电机下发加速度和转速命令。
- 示例 7 使用它分别给四个驱动电机设置不同转速。

参数: set_acc

参数	类型	单位	默认值	说明
motor_id	int	无	必填	电机 ID。当前四个驱动电机为 1、2、3、4。
acc_rad_s2	float	rad/s ²	DEFAULT_ACC_RAD_S2	电机加速度。

参数: set_speed

参数	类型	单位	默认值	说明
motor_id	int	无	必填	电机 ID。当前四个驱动电机为 1、2、3、4。
speed_rad_s	float	rad/s	必填	指定电机目标角速度。

限制范围:

项目	当前范围
speed_rad_s	底层驱动限幅 -44.0 到 44.0 rad/s

返回值:

- 无显式返回值。

```
# 示例 7: 四个驱动电机分别设置不同转速, 运行 1 秒后停车。
# 注意: 这是直接控制电机, 不会自动调整转向舵机角度。
print("示例 7: 四个驱动电机分别设置不同转速, 运行 1 秒后停车。")
motor_speeds = (
    (1, 1.0), # 左前电机 ID=1, 速度 1.0 rad/s
    (2, 0.5), # 右前电机 ID=2, 速度 0.5 rad/s
    (3, -0.5), # 左后电机 ID=3, 速度 -0.5 rad/s
    (4, -1.0), # 右后电机 ID=4, 速度 -1.0 rad/s
)
for motor_id, speed_rad_s in motor_speeds:
    rover.motor_bus.set_acc(motor_id, DEFAULT_ACC_RAD_S2)
    rover.motor_bus.set_speed(motor_id, speed_rad_s)
time.sleep(1)
rover.motor_bus.stop_all(rover.motor_ids)
time.sleep(1)
```

重要注意事项:

- 这是底层电机接口, 不会自动调整转向舵机角度。
- 这是底层电机接口, 不会更新 rover 内部的电机速度缓存。

- 直接 `set_speed` 后，如果要停车，建议使用 `rover.motor_bus.stop_all(rover.motor_ids)`，或逐个 `set_speed(motor_id, 0.0)`。
- 不建议在正常行驶控制中绕过 `rover.drive`。

9. set_steering_angles 六个转向舵机直控接口

接口位置: `servo_control.py`, `ServoControl.set_steering_angles`

调用形式:

`rover.servo_control.set_steering_angles(left_front, left_middle, left_rear, right_front, right_middle, right_rear)`

功能:

- 直接控制六个底盘转向舵机角度。
- 示例 8 使用它让六个转向舵机分别转到不同角度，再回正。

参数:

参数	类型	单位	默认值	说明
<code>left_front</code>	<code>float</code>	<code>deg</code>	必填	左前转向舵机角度。
<code>left_middle</code>	<code>float</code>	<code>deg</code>	必填	左中转向舵机角度。
<code>left_rear</code>	<code>float</code>	<code>deg</code>	必填	左后转向舵机角度。
<code>right_front</code>	<code>float</code>	<code>deg</code>	必填	右前转向舵机角度。
<code>right_middle</code>	<code>float</code>	<code>deg</code>	必填	右中转向舵机角度。
<code>right_rear</code>	<code>float</code>	<code>deg</code>	必填	右后转向舵机角度。

参数顺序:

左前、左中、左后、右前、右中、右后。

限制范围:

项目	当前范围
每个转向角	-90.0 到 90.0 deg

返回值:

- 无显式返回值。

```
# 示例 8: 六个转向舵机分别设置不同角度。
# 参数顺序: 左前、左中、左后、右前、右中、右后。
# print("示例 8: 六个转向舵机分别设置不同角度。")
rover.servo_control.set_steering_angles(
    20.0, 0.0, -20.0,
    -20.0, 0.0, 20.0,
)
time.sleep(1)
rover.center_chassis_servos()
time.sleep(1)
```

注意事项:

- 该接口只控制转向舵机，不控制驱动电机。
- 正常行驶建议使用 `rover.drive`，由 `rover.drive` 同时处理转向和电机速度。
- 调试完成后建议调用 `rover.center_chassis_servos()` 回正。

10. center_chassis_servos 转向舵机回正接口

接口位置: `chassis_control.py`, `LunarRover.center_chassis_servos`

调用形式:

`rover.center_chassis_servos()`

功能:

- 将六个底盘转向舵机全部回到 0 度。

返回值:

- 无显式返回值。

内部行为:

- 该接口也使用 `rover` 内部缓存。
- 如果六个转向舵机已经是 0 度，会跳过重复下发。
- 如果实际下发了新角度，会等待约 20 ms。

```
# 示例 8: 六个转向舵机分别设置不同角度。
# 参数顺序: 左前、左中、左后、右前、右中、右后。
# print("示例 8: 六个转向舵机分别设置不同角度。")
rover.servo_control.set_steering_angles(
    20.0, 0.0, -20.0,
    -20.0, 0.0, 20.0,
)
time.sleep(1)
rover.center_chassis_servos()
time.sleep(1)
```

使用场景:

- 转向舵机单项测试结束后回正。
- 原地转向或特殊舵机姿态结束后恢复普通行驶姿态。

11. set_arm_joint_angles 机械臂四关节绝对角接口

接口位置: `servo_control.py`, `ServoControl.set_arm_joint_angles`

调用形式:

`rover.servo_control.set_arm_joint_angles(roll, pitch1, pitch2, pitch3)`

功能:

- 直接设置机械臂四个关节的目标角度。
- 示例 9 使用该接口设置一个固定机械臂姿态。

参数:

参数	类型	单位	默认值	说明
roll	float	deg	必填	Roll 目标角。
pitch1	float	deg	必填	Pitch1 目标角。
pitch2	float	deg	必填	Pitch2 目标角。
pitch3	float	deg	必填	Pitch3 目标角。

关节限位:

关节	当前范围
Roll	-180.0 到 180.0 deg
Pitch1	-90.0 到 90.0 deg
Pitch2	-150.0 到 150.0 deg
Pitch3	-150.0 到 150.0 deg

返回值:

- 无显式返回值。

内部行为:

- 每个目标角都会自动限幅到对应关节范围内。

```
# 示例 9: 直接设置机械臂四个关节的目标角度。
# 参数顺序: Roll、Pitch1、Pitch2、Pitch3。
# 这是直接舵机角度控制, 适合做固定姿态演示。
print("示例 9: 直接设置机械臂四个关节的目标角度。")
rover.servo_control.set_arm_joint_angles(
    0.0, 50.0, -140.0, 0.0,
)
time.sleep(1)
```

注意事项:

- 这是底层舵机功能接口控制绝对位置, 不会更新 RobotArm 内部维护的 roll_deg、pitch1_deg、pitch2_deg、pitch3_deg。
- 如果确实先用 set_arm_joint_angles 做了直接控制, 再要回到 jog_joints, 需要调用 rover.arm.sync_from_servos() 同步一次。

12. read_arm_joint_angles 机械臂四关节读角接口

接口位置: servo_control.py, ServoControl.read_arm_joint_angles

调用形式:

```
angles = rover.servo_control.read_arm_joint_angles()
```

功能:

- 轮询读取机械臂四个关节舵机的当前角度。

- 示例 10 使用该接口打印四个关节角度。

参数:

- 无。

返回值:

返回一个字典:

字段	说明
roll	Roll 当前角度, 读取失败时为 None。
pitch1	Pitch1 当前角度, 读取失败时为 None。
pitch2	Pitch2 当前角度, 读取失败时为 None。
pitch3	Pitch3 当前角度, 读取失败时为 None。

示例:

```
# 示例 10: 读取机械臂四个关节角度。
print("示例 10: 读取机械臂四个关节角度。")
print(rover.servo_control.read_arm_joint_angles())
time.sleep(1)
```

注意事项:

- 多舵机读角度需要等待回复报文, 会占用 UART 总线时间。
- 不建议在高频遥控循环里每轮都读角度, 否则会明显降低控制频率。
- 调试、上电校准或动作结束后确认角度时再读比较合适。

13. 预留舵机接口

接口位置: servo_control.py, ServoControl.set_reserve_servo_angle、ServoControl.read_reserve_servo_angle

调用形式:

```
rover.servo_control.set_reserve_servo_angle(servo_id, angle_deg)
angle = rover.servo_control.read_reserve_servo_angle(servo_id)
```

功能:

- 控制或读取预留舵机。
- 示例 11 使用它测试预留舵机。
- 默认情况下预留舵机未启用。

启用条件:

- robot_config.py 中 RESERVE_SERVO_ENABLED 需要设置为 True。
- RESERVE_SERVO_IDS 中需要填写实际安装的预留舵机 ID。
- 对应 ID 需要有方向、初始角、最小角、最大角配置。

参数: set_reserve_servo_angle

参数	类型	单位	默认值	说明
servo_id	int	无	必填	预留舵机 ID。
angle_deg	float	deg	必填	预留舵机目标角。

参数: read_reserve_servo_angle

参数	类型	单位	默认值	说明
servo_id	int	无	必填	预留舵机 ID。

返回值: set_reserve_servo_angle

返回值	说明
True	预留舵机已启用, 命令已下发。
False	预留舵机未启用, 命令未下发。

返回值: read_reserve_servo_angle

返回值	说明
float	读取成功, 返回当前角度。
None	预留舵机未启用或读取失败。

```
# 示例 11: 预留舵机测试。
# 需先在 robot_config.py 设置 RESERVE_SERVO_ENABLED = True, 并在 RESERVE_SERVO_IDS 中填写 ID。
print("示例 11: 预留舵机测试。")
if not RESERVE_SERVO_ENABLED:
    print("预留舵机未启用, 请先在 robot_config.py 设置 RESERVE_SERVO_ENABLED = True。")
else:
    reserve_id = RESERVE_SERVO_IDS[0]
    rover.servo_control.set_reserve_servo_angle(reserve_id, 30.0)
    time.sleep(1)
    angle = rover.servo_control.read_reserve_servo_angle(reserve_id)
    print("预留舵机 ID=%d 当前角度: " % reserve_id, angle)
    rover.servo_control.set_reserve_servo_angle(reserve_id, -2.0)
    time.sleep(1)
```

注意事项:

- 未安装预留舵机时保持 `RESERVE_SERVO_ENABLED=False`。
- 如果启用多个预留舵机, 每个 ID 都要补齐对应配置。
- 预留舵机读角同样会占用 UART 总线时间, 不建议放在高频控制循环中连续读取。

14. pivot_turn 底盘原地转向接口

接口位置: chassis_control.py, LunarRover.pivot_turn

调用形式:

rover.pivot_turn(speed_rad_s)

功能:

- 让小车绕车体中心原地转向。
- 同时控制四个驱动电机和六个转向舵机。

- 六个转向舵机会按原地转向姿态摆位：四个角轮打对角线切线角，中间两轮回正。
- 示例 12 使用该接口以 1.0 rad/s 原地右转 1 秒，然后停车。

参数:

参数	类型	单位	默认值	说明
speed_rad_s	float	rad/s	必填	原地转向轮速。正数右转，负数左转，0 表示回正并停车。

限制范围:

项目	当前范围
speed_rad_s	由 MAX_MOTOR_RPM=200.0 和 PIVOT_SPEED_SCALE=0.3 限制，约为 -6.28 到 6.28 rad/s

返回值:

- 无显式返回值。

```
# 示例 12: 底盘以 1.0 rad/s 原地右转 1 秒，然后停车。
# 原地转向：负数左转，正数右转。
print("示例 12: 底盘以 1.0 rad/s 原地右转 1 秒，然后停车。")
rover.pivot_turn(speed_rad_s=1.0)
time.sleep(1)
rover.stop()
time.sleep(1)
```

注意事项:

- `pivot_turn` 是底盘推荐入口，比直接分别控制电机和转向舵机更安全。
- 调试时应先使用较小速度，例如 0.5 到 1.5 rad/s。
- 原地转向前建议架空车轮，确认周围没有障碍物。
- 如果只想停车，建议调用 `rover.stop()`。
- PS2 遥控中 R2 + 右摇杆左右也使用 `pivot_turn` 实现原地转向。

15. 接口选择建议

底盘行驶:

- 优先使用 `rover.drive`。
- 停车使用 `rover.stop`。
- 只有单独测试电机时才直接使用 `rover.motor_bus.set_speed`。

转向舵机:

- 普通行驶不要直接控制六个转向舵机，使用 `rover.drive`。

- 原地转向使用 `rover.pivot_turn`
- 单独测试舵机角度时使用 `rover.servo_control.set_steering_angles`。
- 测试结束后使用 `rover.center_chassis_servos` 回正。

机械臂：

- 回到固定初始姿态使用 `rover.arm.apply_initial_pose`。
- 遥控点动使用 `rover.arm.jog_joints`。
- 绝对角度控制使用 `rover.servo_control.set_arm_joint_angles`。
- 读角只在调试或同步时使用，不放入高频遥控循环。

相机：

- 绝对角度使用 `rover.servo_control.set_camera_angle`。
- 点动使用 `rover.arm.jog_camera`。

预留舵机：

- 默认关闭。
- 安装新舵机并完成 ID、方向、限位配置后，再使用预留舵机接口。

16. 常见问题

1) 为什么有些接口有返回值，有些没有？

- 组合机械臂接口需要维护内部姿态，所以 `apply_initial_pose` 和 `jog_joints` 返回目标姿态字典。
- 底层电机和舵机下发接口主要负责发送命令，通常没有显式返回值。
- 读取接口会返回读到的角度；读取失败时对应值为 `None`。

2) 为什么 `jog_joints` 不每次读取真实舵机角度？

- 读角会占用 UART 总线时间，放在遥控循环中会降低控制频率。
- 当前策略是用内部状态连续点动，需要校准时再调用 `sync_from_servos`。

为什么 `set_arm_joint_angles` 后再 `jog_joints` 可能状态不一致？

- `set_arm_joint_angles` 是 `servo_control.py` 的直接舵机接口，它不会更新 `RobotArm` 内部状态。
- `jog_joints` 使用的是 `RobotArm` 内部状态。
- 如果混用，建议在切回 `jog_joints` 前调用 `rover.arm.sync_from_servos()`。

3) 为什么直接 `set_speed` 后不能只依赖 `rover.stop`？

- `motor_bus.set_speed` 绕过了 `LunarRover` 的命令缓存。
- `rover.stop` 会根据内部状态判断是否需要下发停车命令。
- 所以直接电机调试后，应直接调用 `motor_bus.stop_all` 或逐个 `set_speed` 为 0。